

Módulo 6: Métodos Estatísticos

e Mineração Não-Supervisionada

1 Objetivos de Aprendizagem

- ✓ **Aplicar** testes de hipótese estatísticos para comparar o desempenho de algoritmos de mineração de dados, interpretando p-valores e erros Tipo I e II.
- ✓ **Calcular** e interpretar medidas de correlação e dependência (Pearson, Spearman, Informação Mútua) para análise exploratória de dados.
- ✓ **Implementar** e comparar algoritmos de agrupamento (K-Means, Hierarchical, DBSCAN, GMM) para descoberta de estrutura em dados não rotulados.
- ✓ **Selecionar** o algoritmo de clustering mais adequado conforme a forma geométrica dos grupos, presença de ruído e dimensionalidade dos dados.
- ✓ **Avaliar** agrupamentos com métricas internas (Silhouette, Davies-Bouldin) e externas (ARI, NMI), compreendendo as limitações de cada abordagem.
- ✓ **Minerar** regras de associação usando os algoritmos Apriori e FP-Growth, calculando suporte, confiança, lift e conviction.
- ✓ **Detectar** anomalias usando métodos estatísticos, baseados em distância e em florestas de isolamento (Isolation Forest).
- ✓ **Identificar** as tendências do estado da arte (2020–2024) em clustering profundo, HDBSCAN e detecção moderna de outliers.

2 Fundamentos Estatísticos para Mineração de Dados

2.1 Testes de Hipótese

Um **teste de hipótese** é um procedimento para decidir, com base em evidências amostrais, se uma afirmação sobre uma população deve ser aceita ou rejeitada.

Componentes de um Teste de Hipótese

- H_0 (**hipótese nula**): A posição padrão a ser testada (ex. “algoritmo A e B têm a mesma acurácia”).
- H_1 (**hipótese alternativa**): O que queremos provar (ex. “algoritmo A é melhor que B”).
- **Nível de significância** α : Probabilidade máxima do Erro Tipo I aceitável (tipicamente $\alpha = 0.05$).
- **p-valor**: Probabilidade de observar um resultado tão extremo quanto o ob-

servado, supondo H_0 verdadeira. Rejeita-se H_0 se $p < \alpha$.

Tipos de Erro:

- **Erro Tipo I (α):** Rejeitar H_0 quando ela é verdadeira (*falso positivo*).
- **Erro Tipo II (β):** Não rejeitar H_0 quando ela é falsa (*falso negativo*).
- **Poder do Teste ($1 - \beta$):** Probabilidade de detectar corretamente o efeito quando H_1 é verdadeira.

Testes Relevantes para Mineração de Dados:

- **Teste t pareado:** Compara médias de dois classificadores nos mesmos folds de validação cruzada. Assume normalidade das diferenças.

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad \bar{d} = \frac{1}{n} \sum_{i=1}^n (a_i - b_i)$$

- **ANOVA (Analysis of Variance):** Compara três ou mais classificadores simultaneamente. Decompõe variância total em variância entre grupos e dentro dos grupos.
- **Teste de Wilcoxon Signed-Rank** (não-paramétrico): Alternativa ao teste t pareado quando os dados não seguem distribuição normal. Ranqueia as diferenças absolutas e testa se as medianas são iguais.
- **McNemar's Test:** Compara dois classificadores pelas matrizes de confusão. Testa se o número de casos onde A acerta e B erra é diferente do número onde B acerta e A erra:

$$\chi^2 = \frac{(b - c)^2}{b + c}$$

onde b = casos em que A acerta e B erra, c = casos em que B acerta e A erra.

⚠ Comparações Múltiplas

Ao comparar k classificadores com múltiplos testes de hipótese, a probabilidade de pelo menos um falso positivo é $1 - (1 - \alpha)^k$. Para 10 comparações com $\alpha = 0.05$, isso é $\approx 40\%$! Use correções como **Bonferroni** ($\alpha' = \alpha/k$) ou o teste de **Friedman** + post-hoc de Nemenyi para comparações múltiplas.

Exemplo Prático 1 — Interpretação de Teste de Hipótese

💡 Cenário: Comparação de Dois Algoritmos de Classificação

Um pesquisador deseja saber se o algoritmo **Random Forest** (RF) tem acurácia superior ao **SVM** em um dataset de detecção de spam. Ele executa os dois modelos em 10 folds de validação cruzada e observa as acurácias a seguir.

Fold	RF (%)	SVM (%)
1	92.1	89.3
2	88.7	86.0
3	91.4	90.1
4	93.0	88.5
5	89.5	87.9
6	94.2	91.6
7	90.3	89.0
8	88.0	85.5
9	92.8	90.4
10	91.0	88.7

Passo 1 - Formular as hipóteses:

- H_0 : Não há diferença entre RF e SVM ($\mu_{\text{RF}} = \mu_{\text{SVM}}$).
- H_1 : RF é melhor que SVM ($\mu_{\text{RF}} > \mu_{\text{SVM}}$) — teste unilateral.
- Nível de significância: $\alpha = 0.05$.

Passo 2 - Escolher o teste adequado:

Como temos dois algoritmos avaliados nos *mesmos* folds (amostras pareadas), utilizamos o **Teste t pareado**.

Passo 3 - Calcular as diferenças e a estatística t :

Fold	RF	SVM	$d_i = \text{RF} - \text{SVM}$
1	92.1	89.3	+2.8
2	88.7	86.0	+2.7
3	91.4	90.1	+1.3
4	93.0	88.5	+4.5
5	89.5	87.9	+1.6
6	94.2	91.6	+2.6
7	90.3	89.0	+1.3
8	88.0	85.5	+2.5
9	92.8	90.4	+2.4
10	91.0	88.7	+2.3
Média $\bar{d}$			2.40
Desvio padrão s_d			0.91

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} = \frac{2.40}{0.91/\sqrt{10}} = \frac{2.40}{0.288} \approx 8.33$$

Passo 4 - Determinar o valor crítico e o p-valor:

Com $n-1 = 9$ graus de liberdade e $\alpha = 0.05$ (unilateral), o valor crítico é $t_{0.05, 9} = 1.833$ (Consultar tabela t).

Como $t_{\text{calculado}} = 8.33 \gg 1.833$, então o p-valor ≈ 0.00001 . Logo p-valor $\ll \alpha$.

💡 Interpretação dos Resultados

- 🗳️ **Decisão:** Rejeitamos H_0 ($p < \alpha$).
- 📊 **Conclusão estatística:** Há evidência estatisticamente significativa de que Random Forest supera SVM neste dataset ($t(9) = 8.33$, $p < 0.001$).
- 💡 **Interpretação prática:** O RF foi, em média, 2.4 pontos percentuais mais acurado que o SVM em todos os 10 folds. A diferença não é aleatória.
- ⚠️ **Atenção:** Significância estatística não implica relevância prática. 2.4% pode ou não ser relevante dependendo do custo do erro no domínio da aplicação.

Exemplo Prático 2 - Teste t Pareado (Passo a Passo)

💡 Cenário: Acurácia de Rede Neural vs Regressão Logística

Um experimento avalia se uma Rede Neural MLP supera a Regressão Logística em 5 folds de validação cruzada em um dataset de diagnóstico médico.

Dados:

Fold	MLP (%)	Reg. Log. (%)	d_i
1	85.0	80.0	+5.0
2	90.0	85.0	+5.0
3	78.0	80.0	-2.0
4	92.0	87.0	+5.0
5	88.0	85.0	+3.0
			$\bar{d}$ 3.20
			s_d 2.77

Hipóteses: $H_0 : \mu_d = 0$ vs $H_1 : \mu_d \neq 0$ (bilateral, $\alpha = 0.05$)

Estatística t :

$$\bar{d} = \frac{5 + 5 - 2 + 5 + 3}{5} = 3.20 \quad s_d = \sqrt{\frac{\sum (d_i - \bar{d})^2}{n - 1}} = \sqrt{\frac{(1.8)^2 + (1.8)^2 + (5.2)^2 + (1.8)^2 + (0.2)^2}{4}} \approx 2.77$$

$$t = \frac{3.20}{2.77/\sqrt{5}} = \frac{3.20}{1.239} \approx 2.58$$

Valor crítico bilateral com $gl(\text{Grau de Liberdade}) = n - 1 = 4$: $t_{0.025, 4} = 2.776$.

Como $t_{\text{calc}} = 2.58 < 2.776$, **não rejeitamos** H_0 ($p \approx 0.06 > 0.05$).

💡 Interpretações Possíveis

- 🚫 **Conclusão:** Não há evidência estatisticamente significativa de que o MLP seja diferente da Regressão Logística com $n = 5$ folds.
- ⚠️ **Problema de poder:** Com apenas 5 folds, o poder do teste é baixo. Aumentar para 10 ou 30 folds poderia revelar diferença significativa.
- 💡 **Recomendação prática:** O resultado é *borderline* ($p = 0.06$). Repetir com mais dados antes de descartar o MLP.

Exemplo Prático 3 - ANOVA (Comparação de Três Algoritmos)

💡 Cenário: Três Classificadores em Validação Cruzada

Três algoritmos: **KNN**, **Decision Tree** e **Naive Bayes** são avaliados em 5 folds. Queremos saber se pelo menos um deles tem desempenho diferente dos demais.

Dados (acurácia em %):

Fold	KNN	Decision Tree	Naive Bayes
1	82.0	78.0	75.0
2	84.0	80.0	76.0
3	81.0	79.0	74.0
4	85.0	82.0	77.0
5	83.0	80.0	73.0
Média $\bar{x}_j$	83.0	79.8	75.0

Hipóteses:

$$H_0 : \mu_{\text{KNN}} = \mu_{\text{DT}} = \mu_{\text{NB}} \quad H_1 : \text{ao menos uma média difere}$$

Cálculo da ANOVA (One-Way):

$$\text{Média global: } \bar{x} = \frac{83.0 + 79.8 + 75.0}{3} = 79.27$$

$$\begin{aligned}
 SS_{\text{entre}} &= n \sum_{j=1}^k (\bar{x}_j - \bar{x})^2 = 5[(83 - 79.27)^2 + (79.8 - 79.27)^2 + (75 - 79.27)^2] \\
 &= 5[13.90 + 0.28 + 18.24] = 5 \times 32.42 = 162.1
 \end{aligned}$$

$$\begin{aligned}
 SS_{\text{dentro}} &= \sum_j \sum_i (x_{ij} - \bar{x}_j)^2 \\
 &= \underbrace{(1 + 1 + 4 + 4 + 0)}_{\text{KNN}=10} + \underbrace{(3.24 + 0.04 + 0.64 + 4.84 + 0.04)}_{\text{DT}=8.8} + \underbrace{(0 + 1 + 1 + 4 + 4)}_{\text{NB}=10} = 28.8
 \end{aligned}$$

Fonte	SS	gl	$MS = SS/gl$	F
Entre grupos	162.1	$k - 1 = 2$	81.05	$F = 81.05/2.40 \approx \mathbf{33.77}$
Dentro grupos	28.8	$N - k = 12$	2.40	
Total	190.9	14		

Valor crítico: $F_{0.05, 2, 12} = 3.89$. Como $F = 33.77 \gg 3.89$, p-valor $\ll 0.001$.

💡 Interpretações Possíveis

- 🗳️ **Decisão:** Rejeitamos H_0 - há diferença significativa entre os algoritmos.
- 📊 **Conclusão:** A ANOVA indica que pelo menos um algoritmo difere dos demais ($F(2, 12) = 33.77, p < 0.001$). A ANOVA *não* diz quais pares diferem.
- 🔍 **Post-hoc necessário:** Aplicar teste de Tukey HSD ou Bonferroni para comparar pares: KNN vs DT, KNN vs NB, DT vs NB.
- 💡 **Interpretação prática:** KNN ($\bar{x} = 83\%$) supera NB ($\bar{x} = 75\%$) por 8 p.p., diferença praticamente relevante além de estatisticamente significativa.

Exemplo Prático 4 - Teste de Wilcoxon Signed-Rank

💡 Cenário: Quando os Dados Não São Normais

Um pesquisador compara dois algoritmos em 6 folds, mas os dados têm valores extremos (outliers) que violam a suposição de normalidade do teste t . Utiliza-se o **Teste de Wilcoxon Signed-Rank** como alternativa não-paramétrica.

Dados (F1-Score em %):

Fold	Alg. A	Alg. B	$d_i = A - B$
1	88.0	85.0	+3.0
2	72.0	79.0	-7.0
3	91.0	88.0	+3.0
4	95.0	90.0	+5.0
5	60.0	55.0	+5.0
6	84.0	83.0	+1.0

Hipóteses: H_0 : mediana das diferenças = 0 vs H_1 : mediana $\neq 0$

Passo a passo do Wilcoxon:

1. Calcular $|d_i|$ e ranquear em ordem crescente (excluindo $d_i = 0$):

Fold	d_i	$ d_i $	Rank	Sinal
6	+1.0	1.0	1	+
1	+3.0	3.0	2,5	+
3	+3.0	3.0	2,5	+
4	+5.0	5.0	4,5	+
5	+5.0	5.0	4,5	+
2	-7.0	7.0	6	-

Nota: ranks 2 e 3 são empatados nos $|d_i| = 3$, então recebem rank médio $(2+3)/2 = 2.5$. Idem para $|d_i| = 5$: $(4+5)/2 = 4.5$.

2. Somar ranks por sinal:

$$W^+ = 1 + 2.5 + 2.5 + 4.5 + 4.5 = 15 \quad W^- = 6$$

3. Estatística de teste: $W = \min(W^+, W^-) = \min(15, 6) = 6$

4. Valor crítico (tabela, bilateral, $n = 6$, $\alpha = 0.05$): $W_{\text{crítico}} = 2$.

Como $W = 6 > 2$, **não rejeitamos** H_0 ($p \approx 0.16 > 0.05$).

💡 Interpretações Possíveis

- **⊗ Decisão:** Não rejeitamos $H_0 \rightarrow$ não há evidência de diferença mediana entre A e B com $n = 6$.
- **⚠ Poder baixo:** Com apenas 6 observações, o teste tem poder reduzido. A diferença pode existir mas não ser detectável com amostra tão pequena.
- **💡 Por que Wilcoxon em vez de t ?**
O fold 2 apresenta queda atípica ($d_2 = -7.0$, possível outlier). O teste t daria $t \approx 1.55$ ($p \approx 0.18$) $\rightarrow$ resultado similar, porém *menos confiável* por violar a suposição de normalidade das diferenças.
- **🔍 Quando usar Wilcoxon:**
 - $n < 30$ e não há garantia de normalidade das diferenças.
 - Presença de outliers que distorcem a média.
 - Dados ordinais (ex.: scores de ranking de sistemas).

⚠ Comparações Múltiplas

Ao comparar k classificadores com múltiplos testes de hipótese, a probabilidade de pelo menos um falso positivo é $1 - (1 - \alpha)^k$. Para 10 comparações com $\alpha = 0.05$, isso é $\approx 40\%$! Use correções como **Bonferroni** ($\alpha' = \alpha/k$) ou o teste de **Friedman** + post-hoc de Nemenyi para comparações múltiplas.

2.2 Correlação e Dependência

Coefficiente de Correlação de Pearson (r): Mede relação linear entre duas variáveis contínuas:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}} \in [-1, 1]$$

$r = 1$ (correlação positiva perfeita), $r = 0$ (sem correlação linear), $r = -1$ (negativa perfeita).

Exemplo Prático — Correlação de Pearson com Dataset Fictício

💡 Cenário: Horas de Estudo vs Nota na Prova

Um professor registra as horas de estudo semanal e a nota final (0–10) de 8 alunos. Queremos saber se há correlação linear entre as duas variáveis.

Dataset:

Aluno	Horas (x_i)	Nota (y_i)	$(x_i - \bar{x})$	$(y_i - \bar{y})$	$(x_i - \bar{x})(y_i - \bar{y})$
1	2.0	4.5	-3.25	-2.88	9.36
2	3.0	5.0	-2.25	-2.38	5.36
3	4.0	5.5	-1.25	-1.88	2.35
4	5.0	6.0	-0.25	-1.38	0.35
5	6.0	7.5	+0.75	+0.12	0.09
6	7.0	8.0	+1.75	+0.62	1.09
7	9.0	9.0	+3.75	+1.62	6.08
8	10.0	9.5	+4.75	+2.12	10.07
Média	$\bar{x} = 5.25$	$\bar{y} = 7.38$			$\sum = 34.75$

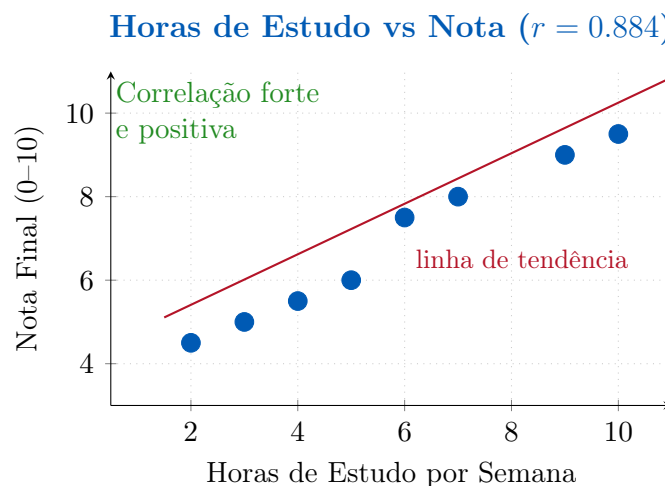
Cálculo passo a passo:

$$\begin{aligned} \sum_{i=1}^8 (x_i - \bar{x})^2 &= (-3.25)^2 + (-2.25)^2 + (-1.25)^2 + (-0.25)^2 + (0.75)^2 + (1.75)^2 + (3.75)^2 + (4.75)^2 \\ &= 10.56 + 5.06 + 1.56 + 0.06 + 0.56 + 3.06 + 14.06 + 22.56 = 57.50 \end{aligned}$$

$$\begin{aligned} \sum_{i=1}^8 (y_i - \bar{y})^2 &= (-2.88)^2 + (-2.38)^2 + (-1.88)^2 + (-1.38)^2 + (0.12)^2 + (0.62)^2 + (1.62)^2 + (2.12)^2 \\ &= 8.29 + 5.66 + 3.53 + 1.90 + 0.01 + 0.38 + 2.62 + 4.49 = 26.88 \end{aligned}$$

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}} = \frac{34.75}{\sqrt{57.50 \times 26.88}} = \frac{34.75}{\sqrt{1545.6}} = \frac{34.75}{39.31} \approx 0.884$$

Visualização:



💡 Interpretações Possíveis

- 📊 **Magnitude:** $r = 0.884$ indica **correlação positiva forte** entre horas de estudo e nota. Quanto mais o aluno estuda, maior tende a ser sua nota.
- 🧠 **Escala de interpretação prática** (Cohen, 1988):
 - $|r| < 0.30$: correlação fraca
 - $0.30 \leq |r| < 0.50$: correlação moderada
 - $|r| \geq 0.50$: correlação forte
- ⚠️ **Correlação $\neq$ Causalidade:** $r = 0.884$ não prova que estudar *causa* nota alta. Um terceiro fator (ex.: dedicação geral do aluno) pode explicar ambas as variáveis.

- 🔍 **Significância estatística:** Para $n = 8$ e $r = 0.884$, a estatística de teste é:

$$t = r \sqrt{\frac{n-2}{1-r^2}} = 0.884 \sqrt{\frac{6}{1-0.781}} = 0.884 \times 5.23 \approx 4.62$$

Com $gl = 6$ e $\alpha = 0.05$, $t_{\text{crítico}} = 2.447$. Como $4.62 > 2.447$, a correlação é **estatisticamente significativa** ($p \approx 0.004$).

- ⚠️ **Limitação:** Pearson só captura relações **lineares**. Se a relação fosse curvilínea (ex.: rendimento cai após muitas horas por exaustão), Pearson subestimaria a dependência. Use Spearman ou Informação Mútua nesses casos.

r	Interpretação	Exemplo prático	Uso recomendado
0.90 a 1.00	Correlação muito forte positiva	Altura vs envergadura	Previsão direta
0.70 a 0.89	Correlação forte positiva	Horas estudo vs nota	Seleção de features
0.50 a 0.69	Correlação moderada positiva	Renda vs consumo	Análise exploratória
0.30 a 0.49	Correlação fraca positiva	Temperatura vs vendas	Cuidado ao usar
0.00 a 0.29	Correlação desprezível	Nº letras nome vs nota	Descartar
< 0	Negativa (mesma escala)	Horas TV vs nota	Idem à positiva

Correlação de Spearman (ρ): Aplica Pearson nos **postos** (*ranks*) dos dados. Detecta monotonicidade, não apenas linearidade. Robusta a outliers e útil para dados ordinais.

Tau de Kendall (τ): Baseado em concordância/discordância de pares. Mais robusto que Spearman para amostras pequenas.

Informação Mútua $I(X; Y)$: Mede dependência geral (não apenas linear) entre variáveis:

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

$I(X; Y) = 0$ iff X e Y são independentes. Muito usado em seleção de features (MRMR).

2.3 Distribuições Relevantes

Tabela 1: Distribuições de probabilidade frequentes em Mineração de Dados.

Distribuição	FDP/FMP	Suporte	Uso em DM
Normal	$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	$\mathbb{R}$	Ruído, GMM, testes paramétricos
Bernoulli	$p^x(1-p)^{1-x}$	$\{0, 1\}$	Classificação binária, Naive Bayes
Binomial	$\binom{n}{k} p^k (1-p)^{n-k}$	$\{0, \dots, n\}$	Contagem de acertos, limiar de confiança
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	$\mathbb{N}_0$	Contagem de eventos raros, detecção de anomalias
Exponencial	$\lambda e^{-\lambda x}$	$[0, +\infty)$	Tempo entre eventos, detecção de outliers
t-Student	$\frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	$\mathbb{R}$	Testes com pequenas amostras
Qui-Quadrado	$\frac{x^{k/2-1} e^{-x/2}}{2^{k/2}\Gamma(k/2)}$	$[0, +\infty)$	Teste de independência (tabelas de contingência)

3 Agrupamento (Clustering)

3.1 Definição e Aplicações

O **agrupamento** (clustering) é uma tarefa de aprendizado não-supervisionado que busca descobrir a estrutura natural dos dados, organizando exemplos similares em grupos (*clusters*) sem o uso de rótulos pré-definidos.

Objetivo formal: Dado $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$, encontrar uma partição $\mathcal{C} = \{C_1, \dots, C_K\}$ tal que objetos dentro de cada C_k sejam similares entre si e dissimilares aos de outros clusters.

Aplicações:

- **Segmentação de clientes:** Grupos de comportamento de compra para marketing direcionado.
- **Detecção de anomalias:** Pontos que não pertencem a nenhum cluster são candidatos a outliers.
- **Sumarização de dados:** Representa o dataset pelos centróides dos clusters (compressão).
- **Modelagem de tópicos:** Agrupa documentos por tema (text mining).
- **Bioinformática:** Agrupamento de genes por padrão de expressão.
- **Compressão de imagens:** K-Means para quantização de cores.

i Avaliação sem Ground Truth

A falta de rótulos torna a avaliação de clustering mais desafiadora do que a classificação supervisionada. Métricas internas (Silhouette, Davies-Bouldin) medem a coesão e separação dos clusters, mas podem favorecer clustering simples. Sempre combine métricas numéricas com inspeção visual (PCA, t-SNE, UMAP) para validar resultados.

3.2 K-Means

O K-Means (MacQueen, 1967; Lloyd, 1982) é o algoritmo de clustering mais popular. Minimiza a **inércia** (soma dos quadrados das distâncias intra-cluster):

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

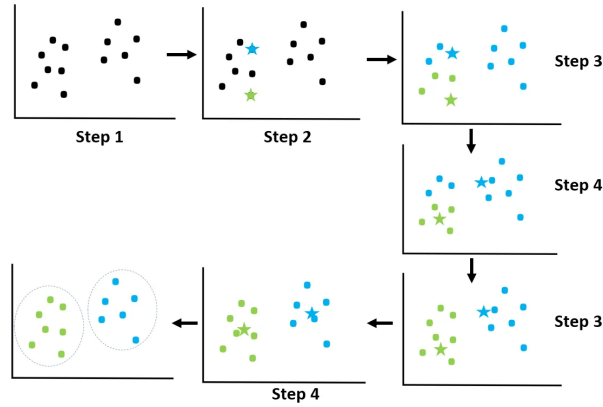
onde $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$ é o centróide do cluster k .

Algorithm 1: K-Means (Algoritmo de Lloyd)**Input:** Dados $\mathcal{X}$, número de clusters K , tolerância ε **Output:** Partição $\{C_1, \dots, C_K\}$ e centróides $\{\mu_1, \dots, \mu_K\}$

```

1 Inicializar centróides  $\mu_1, \dots, \mu_K$  aleatoriamente (ou com K-Means++);
2 repeat
    // Passo de Atribuição (E-step)
3   foreach  $\mathbf{x}_i \in \mathcal{X}$  do
4      $c_i \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \mu_k\|^2$ ;
    // Passo de Atualização (M-step)
5   foreach  $k \in \{1, \dots, K\}$  do
6      $\mu_k \leftarrow \frac{1}{|C_k|} \sum_{\mathbf{x}_i: c_i=k} \mathbf{x}_i$ ;
7 until centróides não mudam (ou  $\Delta < \varepsilon$ );

```

Visualização das etapas do K-Means:**Passo a Passo do Algoritmo K-Means:****1. Definir o número de clusters K .**

Escolha o valor de K de acordo com o problema. Por exemplo, se os dados apresentam dois grupos visíveis, define-se $K = 2$. Quando K não é óbvio, utiliza-se o *Método do Cotovelo* (discutido adiante) para identificar o valor mais adequado.

2. Inicializar os centróides aleatoriamente.

Selecione K pontos de dados aleatoriamente como posições iniciais dos centróides $\mu_1, \mu_2, \dots, \mu_K$. A qualidade da inicialização influencia a convergência; por isso, variantes como K-Means++ melhoram essa etapa.

3. Atribuir cada ponto de dados ao centróide mais próximo.

Para cada ponto $\mathbf{x}_i$, calcula-se a distância a cada centróide e atribui-se o ponto ao cluster cujo centróide é o mais próximo:

$$c_i = \arg \min_k d(\mathbf{x}_i, \mu_k)$$

A medida de distância mais comum é a **distância euclidiana**:

$$d(\mathbf{x}, \mu) = \sqrt{\sum_{j=1}^n (x_j - \mu_j)^2}$$

Exemplo numérico: Considere o ponto $\mathbf{x} = (5, 3)$ e dois centróides $C_1 = (2, 4)$ e $C_2 = (6, 1)$.

$$d(\mathbf{x}, C_1) = \sqrt{(5-2)^2 + (3-4)^2} = \sqrt{9+1} = \sqrt{10} \approx 3.16$$

$$d(\mathbf{x}, C_2) = \sqrt{(5-6)^2 + (3-1)^2} = \sqrt{1+4} = \sqrt{5} \approx 2.24$$

Como $d(\mathbf{x}, C_2) < d(\mathbf{x}, C_1)$, o ponto $(5, 3)$ é atribuído ao cluster C_2 .

4. Recalcular os centróides.

Após a atribuição, cada centróide é movido para a **média** dos pontos pertencentes ao seu cluster:

$$\mu_k \leftarrow \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$$

Exemplo numérico: Suponha que os seguintes 5 pontos foram atribuídos a um cluster:

$$D_1 = (3, 4), D_2 = (4, 1), D_3 = (2, 3), D_4 = (2, 6), D_5 = (3, 3)$$

$$\bar{x} = \frac{1}{5}(3 + 4 + 2 + 2 + 3) = \frac{14}{5} = 2.8$$

$$\bar{y} = \frac{1}{5}(4 + 1 + 3 + 6 + 3) = \frac{17}{5} = 3.4$$

O centróide é movido de sua posição anterior para $(2.8, 3.4)$.

5. Repetir os Passos 3 e 4 até a convergência.

O algoritmo itera os passos de atribuição e recálculo até que os centróides não se movam mais (ou a variação seja menor que um limiar ε). Nesse ponto, cada ponto de dados pertence ao mesmo cluster entre iterações consecutivas e o algoritmo converge.

Escolha de K:

Método do Cotovelo (Elbow Method)

Método do Cotovelo

O Método do Cotovelo consiste em executar o K-Means para diferentes valores de K e plotar a **inércia** (WCSS - Within-Cluster Sum of Squares) em função de K . O ponto onde a redução da inércia desacelera drasticamente forma um “cotovelo” no gráfico, indicando o valor ótimo de K .

$$\text{WCSS}(K) = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

Como funciona:

1. Execute o K-Means para $K = 1, 2, 3, \dots, K_{\max}$.
2. Para cada K , calcule a inércia $\text{WCSS}(K)$.

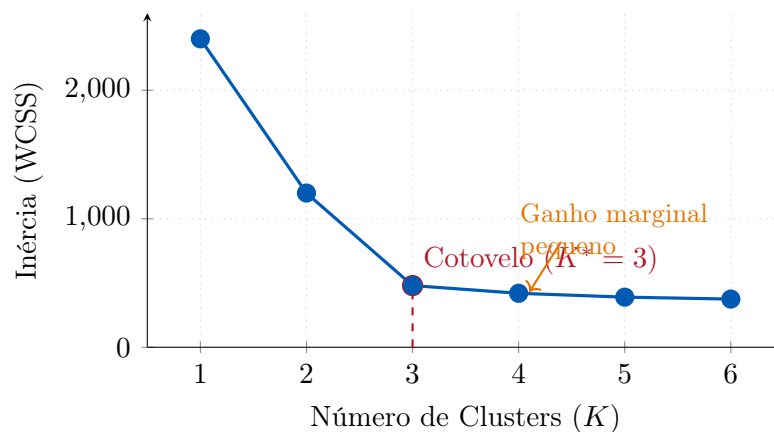
3. Plote a curva $WCSS \times K$.
4. Identifique o ponto de inflexão (cotovelo): onde o ganho marginal de adicionar mais um cluster se torna pequeno.

Exemplo numérico: Dataset com 3 grupos naturais. Inércias calculadas para cada K :

K	WCSS	Redução relativa
1	2400	—
2	1200	−50.0%
3	480	−60.0%
4	420	−12.5%
5	390	−7.1%
6	375	−3.8%

A maior queda ocorre de $K = 2$ para $K = 3$ (−60%). De $K = 3$ em diante, as reduções ficam marginais, indicando que **$K = 3$** é o número ótimo de clusters.

Método do Cotovelo com Inércia vs K



i Limitações do Método do Cotovelo

- O cotovelo pode ser **ambíguo** em datasets com clusters sobrepostos (curva suave sem ponto claro).
- Não considera a forma ou densidade dos clusters, apenas a compacidade interna.
- Combine sempre com a **Análise de Silhouette** para confirmação.

Análise de Silhouette

 Coeficiente de Silhouette

O coeficiente de Silhouette avalia a qualidade do agrupamento para cada ponto, combinando **coesão interna** e **separação externa**. Para cada ponto $\mathbf{x}_i$ no cluster C_k :

$$a(i) = \frac{1}{|C_k| - 1} \sum_{\mathbf{x}_j \in C_k, j \neq i} d(\mathbf{x}_i, \mathbf{x}_j) \quad (\text{distância média intra-cluster})$$

$$b(i) = \min_{l \neq k} \frac{1}{|C_l|} \sum_{\mathbf{x}_j \in C_l} d(\mathbf{x}_i, \mathbf{x}_j) \quad (\text{distância média ao vizinho mais próximo})$$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1] \quad \bar{s} = \frac{1}{N} \sum_{i=1}^N s(i)$$

Interpretação de $s(i)$:

- $s(i) \approx +1$: Ponto bem posicionado no seu cluster, longe dos demais.
- $s(i) \approx 0$: Ponto na fronteira entre dois clusters.
- $s(i) \approx -1$: Ponto provavelmente atribuído ao cluster errado.

Exemplo prático — Cálculo detalhado para P_1 :

Considere 5 pontos distribuídos em 2 clusters: $C_1 = \{P_1, P_2\}$ e $C_2 = \{P_3, P_4, P_5\}$.

Ponto	Coordenadas	Cluster	$s(i)$
P_1	(1.0, 1.0)	C_1	0.74
P_2	(1.5, 2.0)	C_1	0.69
P_3	(3.0, 4.0)	C_2	0.61
P_4	(4.0, 4.5)	C_2	0.66
P_5	(3.5, 5.0)	C_2	0.63

Cálculo passo a passo para $P_1 = (1.0, 1.0)$:

$$a(P_1) = d(P_1, P_2) = \sqrt{(1.5 - 1)^2 + (2 - 1)^2} = \sqrt{1.25} \approx 1.12$$

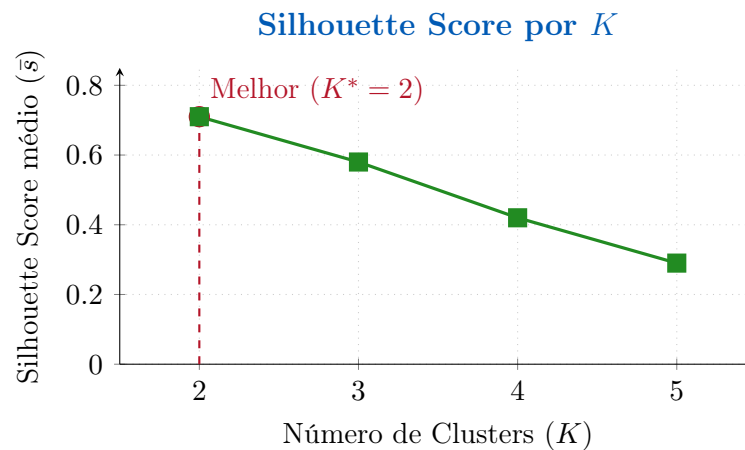
$$\begin{aligned} b(P_1) &= \frac{d(P_1, P_3) + d(P_1, P_4) + d(P_1, P_5)}{3} \\ &= \frac{\sqrt{(3 - 1)^2 + (4 - 1)^2} + \sqrt{(4 - 1)^2 + (4.5 - 1)^2} + \sqrt{(3.5 - 1)^2 + (5 - 1)^2}}{3} \\ &= \frac{\sqrt{13} + \sqrt{21.25} + \sqrt{22.25}}{3} \approx \frac{3.61 + 4.61 + 4.72}{3} \approx 4.31 \end{aligned}$$

$$s(P_1) = \frac{4.31 - 1.12}{\max\{1.12, 4.31\}} = \frac{3.19}{4.31} \approx \mathbf{0.74}$$

$s(P_1) = 0.74$ indica que P_1 está bem posicionado dentro de C_1 e bem separado de C_2 .

Comparação entre valores de K (mesmo dataset):

K	Silhouette Score médio ($\bar{s}$)	Avaliação
2	0.71	Ótimo
3	0.58	Aceitável
4	0.42	Clusters sobrepostos
5	0.29	Agrupamento fraco



i Boas Práticas na Escolha de K

- Use **Método do Cotovelo** e **Silhouette** em conjunto para maior confiabilidade.
- Silhouette Score > 0.5 indica clustering de qualidade razoável; < 0.2 sugere ausência de estrutura.
- O **Gap Statistic** (Tibshirani et al., 2001) é mais robusto estatisticamente: compara a inércia observada com a esperada sob distribuição uniforme de referência.
- Sempre visualize os clusters em 2D (com PCA se necessário) para validação qualitativa.

Variantes do K-Means:

- **K-Means++** (Arthur & Vassilvitskii, 2007): Inicialização melhorada: cada centróide é escolhido com probabilidade proporcional à distância ao centróide mais próximo já escolhido. Garante aproximação $O(\log K)$ do ótimo.
- **K-Medoids (PAM)**: Usa exemplos reais do dataset como centróides (medóides). Mais robusto a outliers.
- **Mini-Batch K-Means**: Processa mini-lotes aleatórios; escala para datasets de milhões de exemplos.

3.3 Clustering Hierárquico

O clustering hierárquico cria uma **hierarquia** de agrupamentos representada em um **dendrograma**, sem necessidade de especificar K antecipadamente.

Agglomerativo (Bottom-up): Começa com N clusters (um por ponto) e mescla iterativamente os dois mais próximos.

Divisivo (Top-down): Começa com um único cluster e divide recursivamente.

CrITÉrios de Ligação (Linkage):

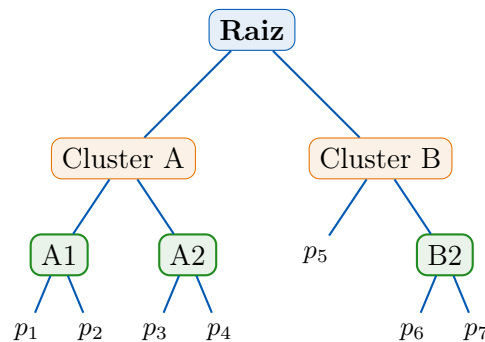
$$d_{\text{single}}(A, B) = \min_{a \in A, b \in B} d(a, b) \quad (\text{single linkage, sensível a outliers}) \quad (1)$$

$$d_{\text{complete}}(A, B) = \max_{a \in A, b \in B} d(a, b) \quad (\text{complete linkage, clusters compactos}) \quad (2)$$

$$d_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(a, b) \quad (\text{UPGMA, balanço}) \quad (3)$$

$$d_{\text{ward}}(A, B) = \Delta J \text{ ao mesclar } A \text{ e } B \quad (\text{Ward, minimiza variância intra-cluster}) \quad (4)$$

Dendrograma (exemplo esquemático):



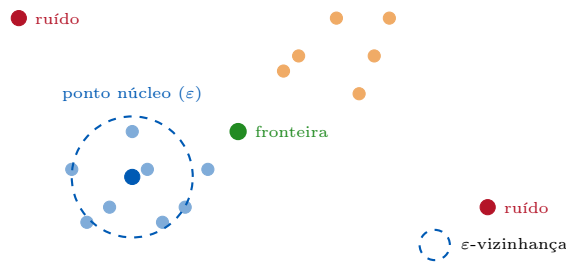
A **correlação cofenética** mede o quão fiel o dendrograma é às distâncias originais; valores > 0.75 indicam boa representação hierárquica.

3.4 DBSCAN

O **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise; Ester et al., 1996) é um algoritmo baseado em densidade que encontra clusters de **forma arbitrária** e detecta **ruído/outliers** automaticamente.

Conceitos: Dado $\varepsilon > 0$ e $\text{MinPts} \in \mathbb{N}$:

- **Ponto Núcleo:** $|N_\varepsilon(\mathbf{x})| \geq \text{MinPts}$, onde $N_\varepsilon(\mathbf{x}) = \{\mathbf{y} : d(\mathbf{x}, \mathbf{y}) \leq \varepsilon\}$.
- **Ponto de Fronteira:** Está na ε -vizinhança de um núcleo, mas não é núcleo.
- **Ponto de Ruído:** Não é núcleo nem fronteira.



DBSCAN: pontos núcleo (azul/laranja), fronteira (verde) e ruído (vermelho)

Algorithm 2: DBSCAN

Input: Dados $\mathcal{X}$, raio ε , densidade mínima MinPts

Output: Rótulos de cluster C para cada ponto (incluindo ruído = -1)

```

1  $C \leftarrow 0$ ;  marcar todos os pontos como não visitados;
2 foreach ponto não visitado  $\mathbf{p} \in \mathcal{X}$  do
3   Marcar  $\mathbf{p}$  como visitado;
4    $N \leftarrow N_\varepsilon(\mathbf{p})$  // vizinhança  $\varepsilon$ 
5   if  $|N| < \text{MinPts}$  then
6     Marcar  $\mathbf{p}$  como ruído;
7   else
8      $C \leftarrow C + 1$ ;
9     ExpandCluster( $\mathbf{p}, N, C, \varepsilon, \text{MinPts}$ );
10 Função ExpandCluster( $\mathbf{p}, N, C, \varepsilon, \text{MinPts}$ ):
11   Adicionar  $\mathbf{p}$  ao cluster  $C$ ;
12   foreach  $\mathbf{q} \in N$  do
13     if  $\mathbf{q}$  não visitado then
14       Marcar  $\mathbf{q}$  como visitado;
15        $N' \leftarrow N_\varepsilon(\mathbf{q})$ ;
16       if  $|N'| \geq \text{MinPts}$  then
17          $N \leftarrow N \cup N'$ ;
18   if  $\mathbf{q}$  ainda não atribuído a cluster then
19     Adicionar  $\mathbf{q}$  ao cluster  $C$ ;
  
```

Parâmetros: ε pode ser estimado pelo gráfico da k -NN distance (joelho da curva para $k = \text{MinPts}$). Regra prática: $\text{MinPts} \geq 2d$ onde d é a dimensionalidade.

3.5 HDBSCAN

O **HDBSCAN** (Hierarchical DBSCAN; McInnes et al., 2017) estende o DBSCAN com uma abordagem hierárquica que **varia o limiar de densidade** automaticamente.

Vantagens sobre DBSCAN:

- Detecta clusters com **densidades variáveis** (problema do DBSCAN: apenas ε global).
- Mais robusto na seleção de parâmetros: apenas `min_cluster_size` é crítico.

- Constrói uma árvore de condensação de clusters e extrai os mais persistentes.
- Retorna probabilidade de pertencimento ao cluster (métrica de membrana).

3.6 Modelos de Mistura Gaussiana (GMM)

O GMM modela os dados como uma **mistura** de K distribuições Gaussianas:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

onde π_k são os pesos da mistura ($\sum_k \pi_k = 1$, $\pi_k > 0$).

O GMM é treinado pelo **algoritmo EM** (Dempster et al., 1977):

E-step: Calcular responsabilidades:

$$r_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step: Atualizar parâmetros:

$$\pi_k = \frac{1}{N} \sum_i r_{ik}, \quad \boldsymbol{\mu}_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{\sum_i r_{ik}}, \quad \boldsymbol{\Sigma}_k = \frac{\sum_i r_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T}{\sum_i r_{ik}}$$

Seleção do número de componentes K : Use **BIC** (Bayesian Information Criterion) ou **AIC**: $\text{BIC} = -2 \log \hat{L} + p \log N$, onde p é o número de parâmetros. Minimize o BIC.

3.7 Spectral Clustering

O Spectral Clustering (Ng et al., 2002) baseia-se na teoria de grafos espectrais para capturar estruturas não-convexas que K-Means falha.

Algoritmo:

1. Construir matriz de similaridade $\mathbf{W}$ (ex. kernel RBF: $w_{ij} = e^{-\|x_i - x_j\|^2 / 2\sigma^2}$).
2. Calcular Laplaciano normalizado: $\mathbf{L} = \mathbf{D}^{-1/2}(\mathbf{D} - \mathbf{W})\mathbf{D}^{-1/2}$, onde $D_{ii} = \sum_j w_{ij}$.
3. Extrair os K menores autovetores de $\mathbf{L}$; formar matriz $\mathbf{U}$.
4. Aplicar K-Means no espaço dos autovetores.

3.8 Avaliação de Agrupamentos

3.8.1 Métricas Internas (sem rótulos verdadeiros)

Coeficiente de Silhouette (Rousseeuw, 1987): Para cada ponto i :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

onde $a(i)$ = distância média intra-cluster, $b(i)$ = distância média ao cluster vizinho mais próximo. $s(i) \in [-1, 1]$: valores próximos a 1 indicam boa alocação.

Índice Davies-Bouldin (Davies & Bouldin, 1979):

$$DB = \frac{1}{K} \sum_{k=1}^K \max_{k \neq k'} \frac{s_k + s_{k'}}{\|c_k - c_{k'}\|}$$

onde s_k é a distância média intra-cluster k . Minimizar DB.

Índice Calinski-Harabasz: Razão entre dispersão inter-cluster e intra-cluster. Maximizar.

3.8.2 Métricas Externas (com rótulos verdadeiros)

Adjusted Rand Index (ARI): Versão ajustada para chance do Rand Index. $ARI \in [-1, 1]$; 1 = agrupamento perfeito; 0 = aleatório.

Normalized Mutual Information (NMI):

$$NMI(U, V) = \frac{2 \cdot I(U; V)}{H(U) + H(V)} \in [0, 1]$$

V-Measure: Média harmônica de homogeneidade (cada cluster contém apenas uma classe) e completude (todos os membros de uma classe estão no mesmo cluster).

4 Regras de Associação

4.1 Conceitos Fundamentais

Dada uma base de transações $\mathcal{T} = \{T_1, \dots, T_N\}$ sobre um conjunto de itens $\mathcal{I}$, o objetivo é encontrar regras $A \Rightarrow B$ ($A, B \subseteq \mathcal{I}$, $A \cap B = \emptyset$) com alta frequência e confiabilidade.

Medidas de interesse:

$$\text{sup}(A) = \frac{|\{T \in \mathcal{T} : A \subseteq T\}|}{|\mathcal{T}|} \quad (\text{frequência do itemset}) \quad (5)$$

$$\text{sup}(A \Rightarrow B) = \text{sup}(A \cup B) \quad (\text{frequência da regra}) \quad (6)$$

$$\text{conf}(A \Rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)} \quad (\text{confiança}) \quad (7)$$

$$\text{lift}(A \Rightarrow B) = \frac{\text{conf}(A \Rightarrow B)}{\text{sup}(B)} \quad (\text{importância relativa}) \quad (8)$$

$$\text{conv}(A \Rightarrow B) = \frac{1 - \text{sup}(B)}{1 - \text{conf}(A \Rightarrow B)} \quad (\text{convicção}) \quad (9)$$

Interpretação:

- **Lift** > 1: A e B são positivamente correlacionados; a regra é mais frequente que o esperado por acaso.
- **Lift** = 1: A e B são independentes.
- **Lift** < 1: A e B são negativamente correlacionados (regra negativa).

💡 Exemplo: Cesta de Mercado

Considere 5 transações: $T_1 = \{\text{leite}, \text{po}\}$, $T_2 = \{\text{leite}, \text{manteiga}, \text{po}\}$, $T_3 = \{\text{manteiga}, \text{queijo}\}$, $T_4 = \{\text{leite}, \text{manteiga}\}$, $T_5 = \{\text{leite}, \text{po}, \text{queijo}\}$.

Para a regra $\{\text{leite}\} \Rightarrow \{\text{po}\}$:

$$\text{sup}(\{\text{leite}, \text{po}\}) = 3/5 = 0.6, \quad \text{sup}(\{\text{leite}\}) = 4/5 = 0.8, \quad \text{conf} = 0.6/0.8 = 0.75$$

$$\text{sup}(\{\text{po}\}) = 3/5 = 0.6, \quad \text{lift} = 0.75/0.6 = 1.25 \quad (\text{correlação positiva})$$

4.2 Algoritmo Apriori (Agrawal & Srikant, 1994)

O Apriori usa a propriedade **anti-monótona**: se um itemset é infrequente, todos os seus superconjuntos também são. Isso permite poda da busca.

Algorithm 3: Apriori

Input: Transações $\mathcal{T}$, suporte mínimo minsup, confiança mínima minconf
Output: Conjunto de regras de associação frequentes

```

1  $L_1 \leftarrow \{\text{itemsets frequentes de tamanho } 1\}$ ;
2  $k \leftarrow 1$ ;
3 while  $L_k \neq \emptyset$  do
4    $C_{k+1} \leftarrow \text{AprioriGen}(L_k)$  // gera candidatos de tamanho  $k+1$ 
5   foreach transação  $T \in \mathcal{T}$  do
6      $D_T \leftarrow \{\text{candidatos em } C_{k+1} \text{ contidos em } T\}$ ;
7     Incrementar contagem de cada itemset em  $D_T$ ;
8    $L_{k+1} \leftarrow \{c \in C_{k+1} : \text{sup}(c) \geq \text{minsup}\}$ ;
9    $k \leftarrow k + 1$ ;
10  $\mathcal{F} \leftarrow \bigcup_k L_k$  // todos os itemsets frequentes
11 foreach  $f \in \mathcal{F}$ ,  $A \subset f$ ,  $B = f \setminus A$ ,  $A \neq \emptyset$  do
12   if  $\text{conf}(A \Rightarrow B) \geq \text{minconf}$  then
13     Adicionar  $A \Rightarrow B$  às regras;
```

4.3 Algoritmo FP-Growth (Han et al., 2000)

O FP-Growth elimina a geração de candidatos construindo uma estrutura compacta (**FP-Tree**) que representa a base de transações.

Vantagens sobre Apriori:

- Apenas **dois varrimentos** da base de dados (independente do número de itemsets).
- Sem geração explícita de candidatos.
- A FP-Tree é tipicamente muito menor que a base original (compartilha prefixos).

Funcionamento: (1) Varre a base, conta frequências de itens, ordena por frequência decrescente. (2) Varre novamente, inserindo transações na FP-Tree. (3) Extrai itemsets frequentes recursivamente usando bases de padrões condicionais.

ECLAT (Equivalence CLAss Transformation): Usa formato vertical dos dados (lista de transações por item, TidList). Interseção de TidLists para computar suporte; eficiente para muitas transações.

4.4 Regras de Associação para Cenários Específicos

- **Regras Quantitativas:** Itens com atributos numéricos; discretização ou particionamento fuzzy.
- **Regras Temporais / Mineração de Sequências:** Regras com ordenação temporal; algoritmo GSP (Generalized Sequential Patterns) e PrefixSpan.
- **Regras Multi-nível:** Taxonomias de itens (ex. leite $\subset$ laticínios $\subset$ alimentos); suporte mínimo ajustado por nível.
- **Regras Negativas:** $A \Rightarrow \neg B$ (comprar A implica não comprar B); úteis para descoberta de anti-correlações.

5 Detecção de Anomalias

Tipos de Anomalias

- **Ponto:** Uma instância individual diverge significativamente das demais (ex. transação fraudulenta).
- **Contextual:** Uma instância é anômala em um contexto específico (ex. temperatura de 35°C em dezembro no RS).
- **Coletiva:** Um subconjunto de instâncias é anômalo em conjunto, mas cada instância individualmente parece normal.

Métodos de Detecção de Anomalias:

- **Estatísticos:** Assume distribuição Gaussiana; Z-score ($|z| > 3$); Grubbs Test; Boxplot ($> 1.5 \times \text{IQR}$).
- **Baseados em Distância:** K-NN Outlier Score: anomalias têm vizinhos distantes.
- **Local Outlier Factor (LOF)** (Breunig et al., 2000): Compara a densidade de um ponto com a de seus vizinhos. $\text{LOF}(p) \gg 1$ indica anomalia.
- **Isolation Forest** (Liu et al., 2008): Constrói árvores de isolamento aleatórias. Anomalias são isoladas em menos partições (caminhos curtos na árvore). Score de anomalia: $s(x, n) = 2^{-E[h(x)]/c(n)}$, onde $h(x)$ é o comprimento do caminho e $c(n)$ é o comprimento médio esperado.
- **Autoencoder:** Treina a rede neural para reconstruir dados normais. Alta **erro de reconstrução** $\|x - \hat{x}\|^2$ indica anomalia.
- **VAE (Variational Autoencoder):** Score de anomalia baseado no ELBO (Evidence Lower Bound).

6 Estado da Arte (2020–2024)

Deep Clustering

Integração de redes neurais profundas com objetivos de clustering:

- **DEC: Deep Embedded Clustering** (Xie et al., 2016): Autoencoder treinado conjuntamente com uma função de perda de clustering; aprende representações otimizadas para agrupamento.
- **SCAN** (Van Gansbeke et al., 2020): Self-supervised clustering; usa similaridade de vizinhos para treinar sem rótulos; state-of-the-art em clustering de imagens.
- **DINO** (Caron et al., 2021): Recursos do Vision Transformer (ViT) pré-treinados auto-supervisionados revelam estrutura de clustering sem fine-tuning; propriedade emergente dos Transformers.

HDBSCAN e Clustering para Dados de Alta Dimensão

- **HDBSCAN** tornou-se o padrão para clustering robusto em Python (scikit-learn ≥ 1.3). Suporta dados de alta dimensão com a métrica de alcançabilidade mútua.
- **Pipeline UMAP + HDBSCAN**: UMAP reduz dimensionalidade preservando estrutura local, HDBSCAN agrupa no espaço reduzido. Muito eficaz para NLP (embeddings de documentos), genômica single-cell (scRNA-seq).
- **BERTopic** (Grootendorst, 2022): Modela tópicos combinando BERT embeddings + UMAP + HDBSCAN + TF-IDF por cluster. Supera LDA em qualidade de tópicos.

Detecção de Outliers Moderna

- **PyOD** (Zhao et al., 2019): Biblioteca Python com 40+ algoritmos de detecção de outliers unificados em uma API sklearn-compatível.
- **COPOD** (Li et al., 2020): Copula-Based Outlier Detection; sem hiperparâmetros; muito eficiente; baseado em cópulas empíricas.
- **ECOD** (Li et al., 2022): Empirical Cumulative distribution-based Outlier Detection; computacionalmente eficiente; interpretável; sem hiperparâmetros.
- **Detecção de Anomalias em Grafos**: GraphSAGE + Isolation Forest para detecção de fraudes em redes sociais; muito relevante para Social CRM.

7 Exercícios

1. **Silhouette Manual:** Dados os pontos $p_1 = (1, 1)$, $p_2 = (2, 1)$, $p_3 = (5, 5)$, $p_4 = (6, 5)$ e os clusters $C_1 = \{p_1, p_2\}$, $C_2 = \{p_3, p_4\}$, calcule o coeficiente de silhouette para cada ponto e a silhouette média. O clustering é bom?
2. **K-Means com Diferentes K:** Aplique K-Means com $K = 2, 3, 4, 5$ ao dataset Iris (4 features). Para cada K , calcule a inércia, Silhouette e Davies-Bouldin. Plote as curvas e determine o K ótimo. O K ótimo coincide com o número real de classes (3)?
3. **Comparação de Linkages:** Aplique clustering hierárquico com single, complete, average e Ward linkage ao dataset Wine. Para cada método, corte o dendrograma em $K = 3$ e calcule ARI e NMI em relação às classes verdadeiras. Qual linkage tem melhor desempenho? Por quê?
4. **Apriori Manual:** Dada a base de transações: $T_1 = \{A, B, C\}$, $T_2 = \{A, B\}$, $T_3 = \{B, C, D\}$, $T_4 = \{A, C\}$, $T_5 = \{A, B, D\}$, $T_6 = \{B, C\}$. Com suporte mínimo = 0.5 e confiança mínima = 0.7, execute o Apriori: liste os itemsets frequentes L_1, L_2, L_3 e as regras válidas com suporte, confiança e lift.
5. **DBSCAN Parameter Selection:** Aplique DBSCAN ao dataset `make_moons` de scikit-learn. Varie $\epsilon \in \{0.1, 0.2, 0.3, 0.5\}$ e $\text{MinPts} \in \{3, 5, 10\}$. Para cada combinação, reporte número de clusters, número de pontos de ruído e ARI. Qual combinação é melhor? Compare com K-Means ($K = 2$).
6. **Comparação de Métodos de Anomalia:** Usando o dataset Pima Indians Diabetes ou Credit Card Fraud, compare Isolation Forest, LOF e Z-Score para detecção de anomalias. Avalie com AUC-ROC e Precisão@K ($K = 1\%$ dos dados). Qual método tem melhor desempenho?
7. **Avaliação de Clustering Pipeline:** Implemente um pipeline completo: (1) normalizar dados, (2) reduzir com PCA para 2D, (3) aplicar K-Means e HDBSCAN, (4) visualizar com t-SNE, (5) calcular todas as métricas de avaliação disponíveis. Reporte e discuta os resultados para o dataset Fashion-MNIST (10.000 amostras).

8 Referências

Referências

- [1] Agrawal, R.; Srikant, R. Fast algorithms for mining association rules. *Proceedings of the 20th VLDB Conference*, pp. 487–499, 1994.
- [2] Han, J.; Pei, J.; Yin, Y. Mining frequent patterns without candidate generation. *ACM SIGMOD Record*, 29(2):1–12, 2000.
- [3] MacQueen, J. Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1:281–297, 1967.

- [4] Arthur, D.; Vassilvitskii, S. k-means++: The advantages of careful seeding. *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035, 2007.
- [5] Ester, M.; Kriegel, H.-P.; Sander, J.; Xu, X. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the 2nd ACM International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231, 1996.
- [6] McInnes, L.; Healy, J.; Astels, S. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017.
- [7] Breunig, M. M.; Kriegel, H.-P.; Ng, R. T.; Sander, J. LOF: Identifying density-based local outliers. *ACM SIGMOD Record*, 29(2):93–104, 2000.
- [8] Liu, F. T.; Ting, K. M.; Zhou, Z.-H. Isolation forest. *IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008.
- [9] Rousseeuw, P. J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- [10] Davies, D. L.; Bouldin, D. W. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2):224–227, 1979.
- [11] Dempster, A. P.; Laird, N. M.; Rubin, D. B. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society. Series B*, 39(1):1–38, 1977.
- [12] Ng, A. Y.; Jordan, M. I.; Weiss, Y. On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems*, 14, 2002.
- [13] Zhao, Y.; Nasrullah, Z.; Li, Z. PyOD: A Python toolbox for scalable outlier detection. *Journal of Machine Learning Research*, 20(96):1–7, 2019.
- [14] Caron, M.; Touvron, H.; Misra, I.; et al. Emerging properties in self-supervised vision transformers. *International Conference on Computer Vision (ICCV)*, 2021.